CERTIFIED ETHICAL HACKER | CEH v13

Module 18

IoT and OT Hacking

Practical Lab Report

PREPARED BY	Dhruva
DATE	9 November 2026
COURSE	CEH v13 - Module 18 Labs
LAB DURATION	1 hour 32 minutes
RESULT	4 / 4 - Passed
CLASSIFICATION	Training / Educational Use

All activities were performed in an authorised, isolated CEH lab environment.

Contents

1	Executive Summary
2	Objectives, Scope and Environment
3	Lab 1 - Reconnaissance of Exposed MQTT Services (Whois and Shodan)
4	Lab 2 - MQTT Traffic Capture and Analysis (Bevywise and Wireshark)
5	Lab 3 - OT / Automotive: CAN Bus Sniffing and Replay (ICSim)
6	Consolidated Findings and Risk Rating
7	Countermeasures and Recommendations
8	Assessment Results
9	Conclusion
10	References

1. Executive Summary

This report documents the practical work completed for CEH v13 Module 18 (IoT and OT Hacking). The exercises cover the full lifecycle of an IoT/OT assessment in a controlled lab: passive reconnaissance of internet-exposed MQTT brokers, deployment and monitoring of a simulated IoT temperature-sensor network, packet-level analysis of MQTT traffic, and sniffing and replaying Controller Area Network (CAN) traffic against a simulated vehicle instrument cluster.

The labs show that the protocols underpinning IoT and OT systems were designed for reliability and low overhead rather than security. MQTT sessions in the lab ran over TCP port 1883 with authentication disabled and no transport encryption, so message topics and payloads were readable in clear text. CAN bus frames carry no sender authentication, which allowed captured traffic to be replayed to manipulate the simulator. Shodan results demonstrated that the same weaknesses are visible at internet scale.

Key outcomes

- Identified the MQTT default port (**1883**) and used it to enumerate exposed brokers with Shodan.
- Confirmed the Bevywise MQTT Route / IoT Simulator WebSocket port (**10443**).
- Captured and dissected the MQTT keep-alive and QoS 2 publish handshake in Wireshark.
- Created a virtual CAN interface (**vcan0**), logged traffic with candump and replayed it with canplayer.
- Passed the module assessment with a score of **4 / 4**.

2. Objectives, Scope and Environment

Objectives

- Understand the IoT and OT attack surface and the information attackers can gather passively.
- Use search engines for IoT devices (Shodan) to locate hosts with MQTT enabled.
- Capture and analyse traffic between an MQTT broker and an IoT device.
- Sniff, log and replay CAN bus traffic to demonstrate a replay attack on an automotive system.
- Map observed weaknesses to practical countermeasures.

Scope and authorisation

All active testing (MQTT broker, simulator, Wireshark capture and ICSim) was performed against systems built for the lab inside a private 10.10.1.0/24 network. Shodan and Whois activity was limited to passive, publicly indexed information. No third-party host was probed, accessed or modified, and no credentials were attempted against any system outside the lab.

Lab environment

Component	Details
Network	Private lab subnet 10.10.1.0/24 (Ubuntu VM on 10.10.1.9; MQTT device 10.10.1.22; broker host 10.10.1.19)
IoT platform	Bevywise MQTT Route (broker + dashboard) and Bevywise IoT Simulator, network *CEH_FINANCE_NETWORK*
Analysis	Wireshark (display filter: `mqtt`)
OT / automotive	OpenGarages ICSim on Ubuntu 22.04 (Jammy) with can-utils 2020.11.0-1
Reconnaissance	Shodan search engine; Whois lookup service (oasis-open.org)
Browser / OS	Firefox on Windows 11 (analysis host); Ubuntu virtual machine (CAN labs)

3. Lab 1 - Reconnaissance of Exposed MQTT Services

3.1 Background

MQTT (Message Queuing Telemetry Transport) is a lightweight publish/subscribe protocol widely used by IoT devices. Brokers conventionally listen on TCP **1883** (unencrypted) and **8883** (TLS). A broker reachable from the internet without authentication allows anyone to subscribe to topics, including the broker's own $SYS statistics topics.

3.2 Whois footprinting

A Whois lookup of **oasis-open.org** (the standards body that maintains MQTT) was used to practise domain footprinting. Registration data is public and useful for mapping an organisation's infrastructure and identifying DNS providers.

Field	Observed value
Domain	oasis-open.org
Registered on	1998-03-04
Expires on	2027-03-03
Last updated	2026-08-12
Status	clientDeleteProhibited, clientTransferProhibited, clientUpdateProhibited
Name servers	karl.ns.cloudflare.com, rita.ns.cloudflare.com
Registrar	DNC Holdings, Inc.

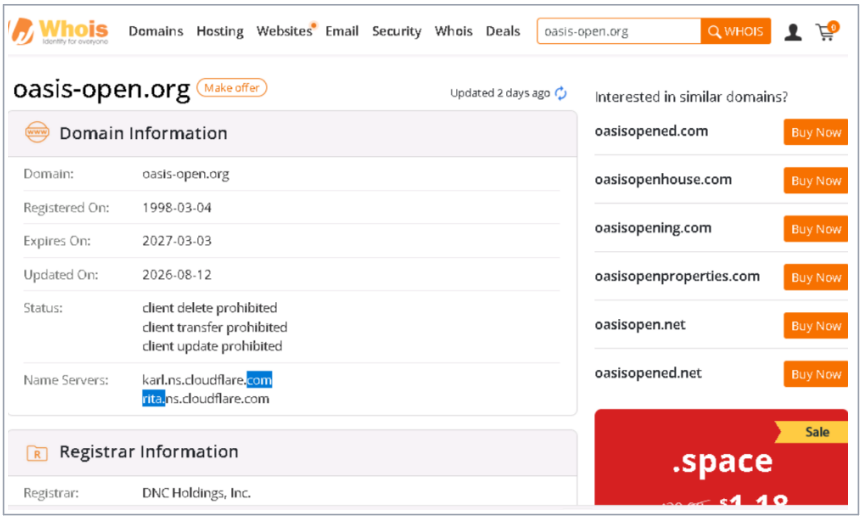


Figure 1: Whois lookup showing registration dates, status flags and name servers.

Observation: the registrar lock flags reduce the risk of unauthorised domain transfer or modification, and the use of Cloudflare name servers indicates the domain sits behind a CDN/DNS provider that also masks origin infrastructure.

3.3 Shodan search for MQTT

Searching Shodan with the MQTT port number (**port:1883**) returned roughly 790,000 indexed results. The facet panels summarise where exposure is concentrated.

Top countries	Hosts	Top organisations	Hosts
China	313,591	Aliyun Computing Co., Ltd	172,883
Korea, Republic of	139,155	SK Broadband Co Ltd	130,062
United States	73,741	Aliyun Computing Co.LTD	54,277
Singapore	45,457	Fly.io, Inc.	31,509
United Kingdom	34,079		

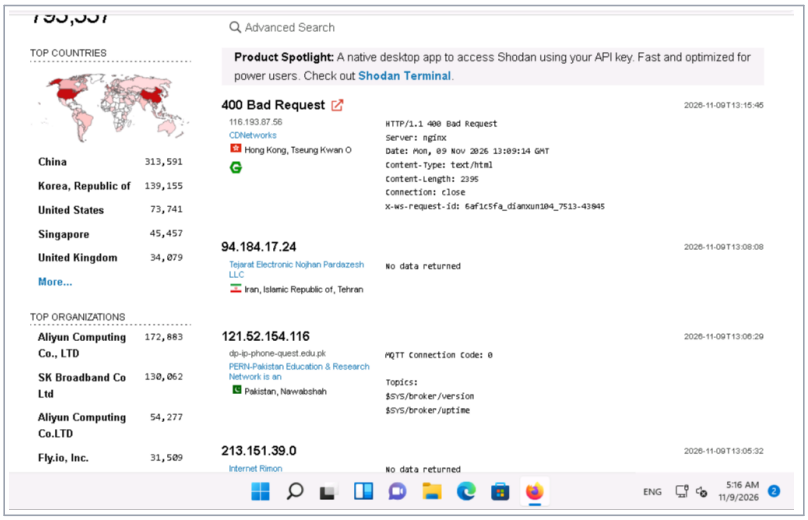


Figure 2: Shodan search results with top countries, top organisations and banner snippets.

Observations

- Result banners vary: some hosts return an MQTT response (*MQTT Connection Code: 0*, meaning the connection was accepted, with `$SYS/broker/version` and `$SYS/broker/uptime` topics listed), while others return no data or an HTTP *400 Bad Request* from an nginx front end.
- A connection code of 0 on a public broker indicates anonymous access is accepted - the broker discloses version and uptime to any client, which assists targeted attacks.
- Concentration in cloud providers shows that misconfigured brokers are frequently deployed on public cloud virtual machines without access controls.

3.4 Host detail and implied vulnerabilities

Inspecting a Shodan host page showed an nginx web server with the default welcome page and open **portmapper (RPC) on port 111** over both TCP and UDP, advertising versions 2, 3 and 4. Shodan lists vulnerabilities as *implied* - inferred from software and version banners rather than verified by testing.

CVE	CVSS	Summary
CVE-2025-23419	5.3 (Medium)	When multiple nginx server blocks share one IP address and port, TLS session resumption (session tickets / session cache) can be used to bypass client-certificate authentication.

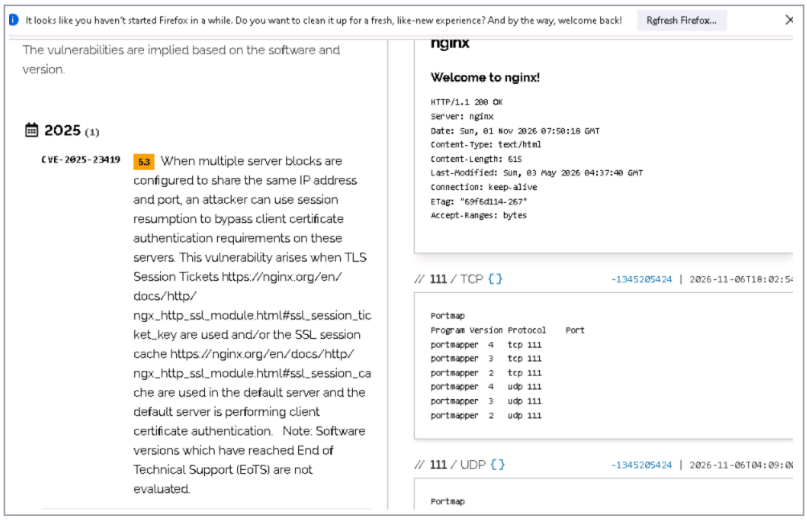


Figure 3: Shodan host detail with implied CVE and exposed portmapper service.

Observation: exposed RPC services and default web pages indicate an unhardened host. Because the CVE is inferred from banners, it must be validated with authorised testing before it is reported as confirmed.

4. Lab 2 - MQTT Traffic Capture and Analysis

4.1 Setup

A Bevywise MQTT Route broker and the Bevywise IoT Simulator were deployed on the lab network. A simulated temperature-sensor device (client ID **TS1**) was created under the network *CEH_FINANCE_NETWORK*. The simulator reaches the broker over a WebSocket connection on port **10443**, while the broker's standard MQTT listener uses TCP port **1883**.

Setting	Value observed
Device name / Client ID	Tempurature_sensor (as named in the lab) / TS1
Device description	Temperature sensor IoT device created for conference purposes
Authentication	**Disabled**
Will message	Disabled
Broker dashboard	Active devices 1; total devices 1; recent connection TS1 from 10.10.1.22 at 05:34:24
Test event	Command received by TS1 at 05:47:23: topic `High_Tempe`, message "Alert for High temprature"

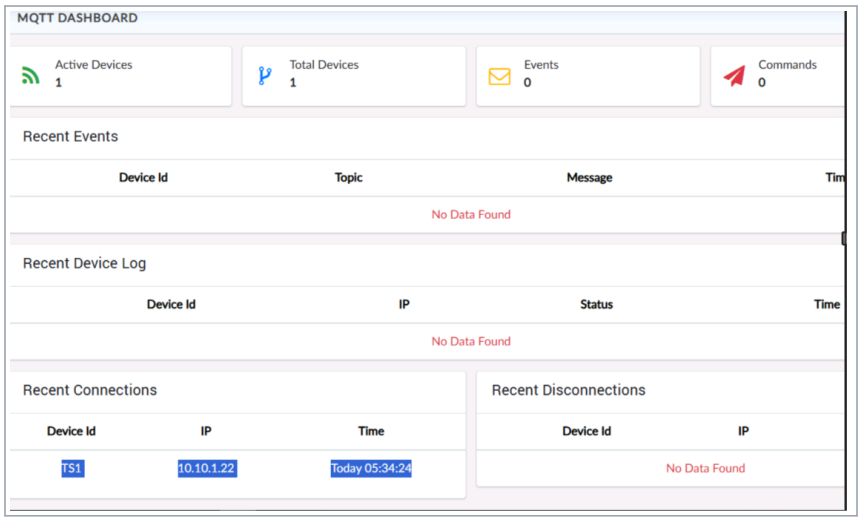


Figure 4: MQTT dashboard showing device TS1 connected from 10.10.1.22.

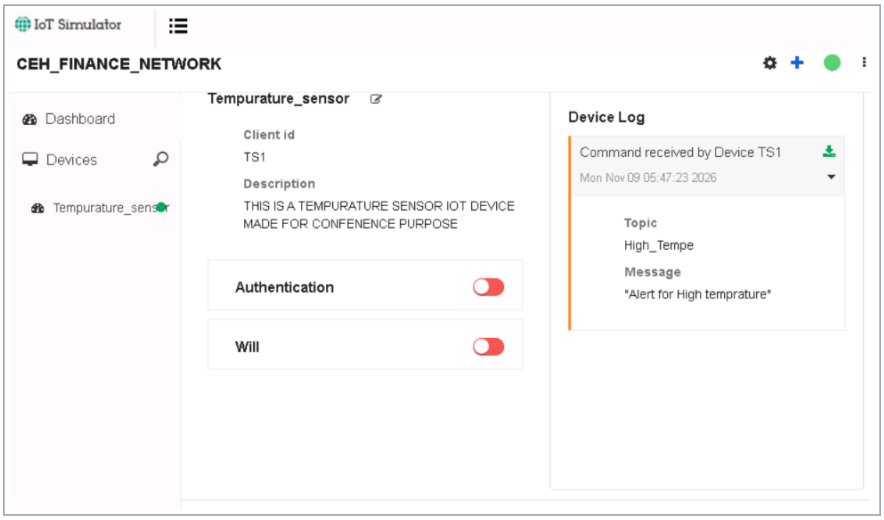


Figure 5: IoT Simulator showing TS1 with authentication disabled and the received command in the device log.

4.2 Wireshark capture

Wireshark was started with the display filter `mqtt` to isolate MQTT traffic between the device (10.10.1.22) and the broker host (10.10.1.19).

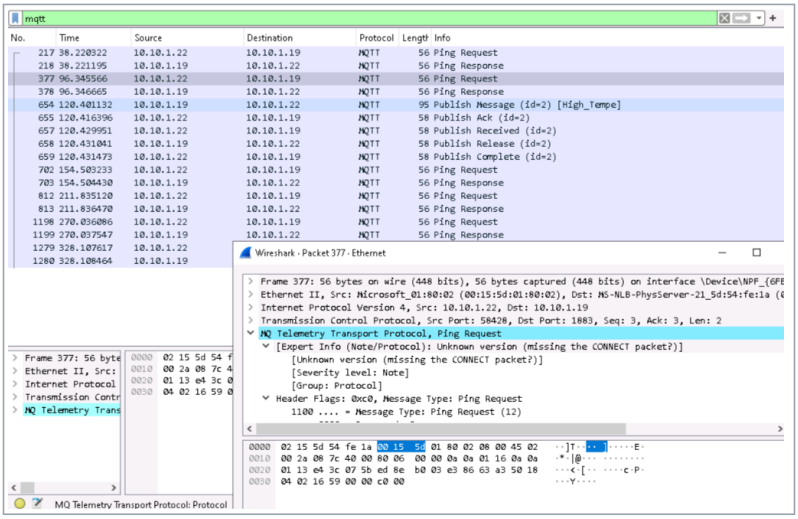


Figure 6: Wireshark MQTT capture with the PINGREQ packet 377 expanded.

Observation	Evidence
Keep-alive heartbeat	PINGREQ / PINGRESP pairs recurring roughly every 58 seconds (packets 217/218, 377/378, 702/703, 812/813, 1198/1199, 1279/1280)
QoS 2 delivery	Packets 654-659: Publish Message (id=2) on topic High_Tempe, then Publish Received, Publish Release and Publish Complete - the four-step exactly-once handshake
Port in use	TCP destination port **1883** (packet 377: 10.10.1.22 to 10.10.1.19)
Expert info	"Unknown version (missing the CONNECT packet?)" - the capture began after the session was established, so the CONNECT packet was not seen
Confidentiality	Topic names and payloads visible in clear text; no TLS

Analysis: because MQTT on 1883 is unencrypted and the device was configured without authentication, any host able to observe traffic on the segment can read topics and payloads. An attacker with network access could also subscribe to topics, publish forged messages (for example a false "High temperature" alert) or hijack a client ID. The expert-info note also shows that analysts who miss the CONNECT packet lose protocol-version context, which is a useful reminder to start captures before sessions are established.

5. Lab 3 - OT / Automotive: CAN Bus Sniffing and Replay

5.1 Background

The Controller Area Network (CAN) bus connects electronic control units in vehicles and industrial equipment. CAN is a broadcast protocol: every node sees every frame, frames are identified by an arbitration ID rather than a sender address, and there is no built-in authentication or encryption. This makes sniffing and replay attacks straightforward once an attacker has bus access.

5.2 Virtual CAN interface and tooling

The can-utils package was installed on Ubuntu 22.04 and a virtual CAN interface was created so the ICSim simulator could be tested without physical hardware:

```
sudo apt install can-utils
sudo modprobe can
sudo modprobe vcan
sudo ip link add dev vcan0 type vcan
sudo ip link set up vcan0
ifconfig
```

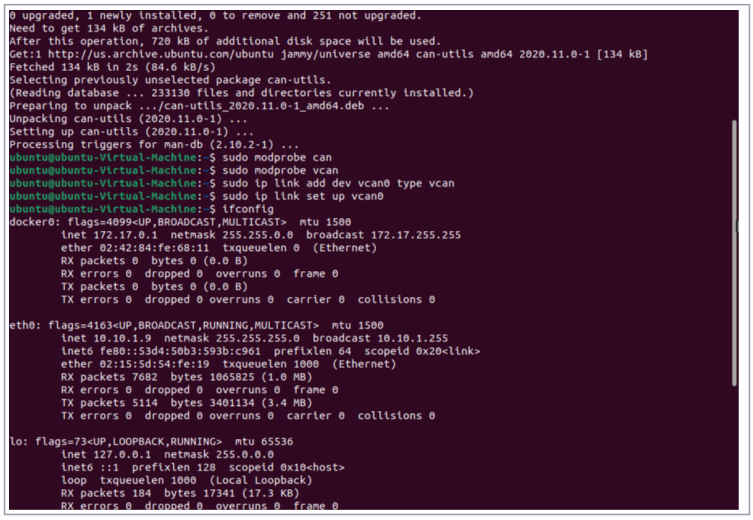


Figure 7: Installing can-utils and bringing up the virtual CAN interface vcan0.

Interface used for sniffing: vcan0. The `ifconfig` output also confirms the VM's eth0 address of 10.10.1.9/24 alongside the Docker bridge (172.17.0.1).

5.3 Sniffing CAN traffic

`cansniffer` was run against vcan0, displaying one row per arbitration ID and highlighting in red the bytes that changed. The header shows the active settings (`l=20 h=100 t=500 slots=36`). Operating the ICSim controls (accelerate, turn left/right, lock/unlock doors) produced changes on specific IDs, which is how an analyst correlates a function with its CAN message.



Figure 8: cansniffer showing CAN IDs, data bytes and changed bytes.

5.4 Logging and replay attack

Traffic was recorded with `candump -l vcan0`, which wrote a timestamped log file (`candump-2026-11-09_091444.log`) into the ICSim directory. The ICSim *CANBus Control Panel* (Accelerate = Up Arrow, Unlock = Right Shift, Lock = Left Shift) and the *IC Simulator* instrument cluster were used to generate and observe activity. A previously captured log was then replayed with `canplayer -I <logfile>`, re-injecting the recorded frames onto the bus so the instrument cluster reproduced the original actions.

```
candump -l vcan0
canplayer -I candump-2026-04-29_060452.log
```

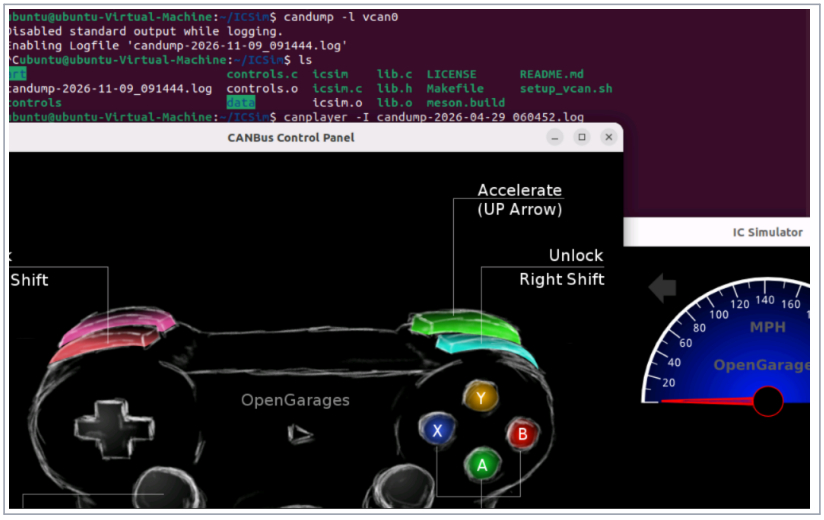


Figure 9: Logging with candump, listing the log file, and replaying with canplayer while ICSim runs.

Analysis: replayed frames are processed as if genuine because CAN has no authentication, freshness check or sender verification, so the instrument cluster cannot tell a replay from live traffic. In a real vehicle or plant network, the same technique could repeat commands such as unlocking doors or changing speed readings if an attacker gains access to the bus (for example through the OBD-II port or a compromised ECU).

6. Consolidated Findings and Risk Rating

Ratings are qualitative judgements based on the lab observations and typical real-world impact.

#	Finding	Evidence	Risk
F1	MQTT service exposed with anonymous access accepted	Shodan banners: connection code 0, $SYS topics disclosed	High
F2	MQTT authentication disabled on device TS1	IoT Simulator: Authentication toggle off	High
F3	MQTT traffic unencrypted on TCP 1883	Wireshark: readable topic High_Tempe and payload	High
F4	No message authentication or replay protection on CAN bus	Logged frames re-injected onto vcan0 with canplayer	High
F5	Implied nginx vulnerability CVE-2025-23419 (CVSS 5.3)	Shodan host detail, banner-based	Medium
F6	Portmapper (RPC) exposed on TCP/UDP 111	Shodan host detail	Medium
F7	Public registration and DNS information available for footprinting	Whois: registrar, dates, name servers	Low

7. Countermeasures and Recommendations

MQTT and IoT

- Disable anonymous access; require unique credentials or client certificates per device.
- Enforce TLS (port 8883) and disable the plaintext listener on 1883 wherever possible.
- Apply topic-level access control lists so a device can only publish or subscribe to its own topics; restrict $SYS/# to administrators.
- Do not expose brokers directly to the internet; use VPNs, private networks or firewall allow-lists.
- Use unique client IDs, enable Last Will where appropriate, and monitor for duplicate or unexpected clients.
- Keep firmware and broker software patched; change default credentials and ports' default configurations.
- Regularly check your own exposure on Shodan and similar services.

Web server and network hygiene

- Patch nginx and review configurations where several server blocks share an IP and port, particularly when client-certificate authentication is used (CVE-2025-23419).
- Disable or firewall RPC portmapper (111) and other unneeded services; remove default welcome pages.

CAN / OT

- Segment safety-critical buses behind a gateway that filters and rate-limits cross-domain messages.
- Introduce message authentication and freshness (for example AUTOSAR SecOC, counters and MACs) so replayed frames are rejected.
- Deploy CAN intrusion detection to flag abnormal message rates, unknown IDs and replay patterns.

- Restrict physical access to diagnostic ports (OBD-II) and secure diagnostic sessions with authentication.
- For industrial OT, apply network segmentation (IT/OT zones), allow-listing and continuous monitoring.

8. Assessment Results

The module assessment (CEHv13 - Module 18: IoT and OT Hacking, duration 1 hour 32 minutes) was completed with a score of **4 / 4**.

#	Question (summary)	Answer	Result
1	Port number to enter in Shodan to find hosts with MQTT enabled	1883	Correct
2	Default TCP port used by Bevywise MQTT Route to connect with Bevywise IoT Simulator	1883	Correct
3	Default WebSocket port used by Bevywise MQTT IoT Simulator to connect with MQTT Route	10443	Correct
4	Interface used while sniffing CAN traffic in the Ubuntu ICSim lab	vcan0	Correct

9. Conclusion

The Module 18 labs showed how weak default configurations in IoT and OT protocols translate into practical attack opportunities. MQTT without authentication or TLS exposes both message content and control of devices, Shodan makes such exposure discoverable at scale, and CAN bus replay demonstrates the lack of authentication in embedded vehicle and industrial networks. Defence relies on layered controls: strong authentication and encryption, network segmentation, patching, message authentication and continuous monitoring. All techniques were practised solely in an authorised lab and should only ever be applied with explicit permission.

10. References

- EC-Council, *Certified Ethical Hacker (CEH) v13*, Module 18: IoT and OT Hacking.
- OASIS, *MQTT Version 3.1.1 / 5.0 Specification*, https://mqtt.org and https://www.oasis-open.org.
- Shodan search engine, https://www.shodan.io.
- National Vulnerability Database, CVE-2025-23419 (nginx TLS session resumption client certificate bypass).
- Bevywise Networks, MQTT Route and IoT Simulator documentation.
- OpenGarages, ICSim - Instrument Cluster Simulator, https://github.com/zombieCraig/ICSim.
- Linux can-utils (candump, cansniffer, canplayer), https://github.com/linux-can/can-utils.
- OWASP, *IoT Top 10*, https://owasp.org/www-project-internet-of-things/.